

## **SSM Site Legend: Feature Engineering & "Vibe Coding" Eradication Blueprint**

**Author:** Sithara Sangeeth SSM (Sithara Sangeeth / @itz\_me\_ssm)

**Platform:** SSM Site Legend | Senior Software Engineering & Productivity Series

**Resource:** Day 03 - Escaping the "Vibe Code" Doom Loop & Building Production-Ready Features

### **1. Executive Overview**

"Vibe coding" is the developer equivalent of social media doom-scrolling. It occurs when engineers rapidly jump from building one feature to the next using AI tools (Cursor, Windsurf, Claude) without completing, securing, or thoroughly testing the previous one. This creates a graveyard of 80%-finished features, bloated codebases, and fragile applications that collapse in production. To build enterprise-grade software, developers must abandon the instant gratification of rapid UI generation and adopt the "3-Day Feature Protocol": a disciplined framework of scoping, implementing, and stress-testing a single feature to 100% completion before writing a single line of code for the next.

### **2. Feature Engineering Master Prompt for AI Tools**

Copy and paste the prompt below into your AI coding tool (Cursor, Claude Code, Windsurf) before starting *any* new feature. This forces the AI to act as a strict Tech Lead rather than a passive code generator.

#### **System Architecture Directive: The 3-Day Feature Protocol & Zero-Vibe Execution**

You are acting as a Strict Senior Principal Engineer. We are abandoning "vibe coding." For the upcoming feature, you must enforce a rigorous, step-by-step engineering lifecycle. Do NOT generate the entire feature at once.

#### **CRITICAL EXECUTION REQUIREMENTS:**

##### **1. PHASE 1 (ARCHITECTURE & DATA SCOPE):**

- Before writing any frontend UI code, explicitly outline the required database schema changes, API endpoint structures, and authentication layers for this specific feature.
- Pause and wait for my explicit approval on the architecture before proceeding.

##### **2. PHASE 2 (CORE IMPLEMENTATION):**

- Write clean, modular, and request-scoped backend logic.
- Strictly forbid placeholder logic, unhandled promises, or "TODO" comments for critical paths. Ensure all database transactions are optimized.

##### **3. PHASE 3 (STRESS TEST & SECURITY VALIDATION):**

- Generate strict edge-case validations and security checks (e.g., IDOR prevention, rate limiting) for this specific feature.
- Provide a QA checklist.

Do NOT suggest, conceptualize, or write code for any outside features until this specific lifecycle is explicitly marked by me as 100% complete and stress-tested.

### **3. Step-by-Step Architectural Implementation (The 3-Day Rule)**

#### **• Checkpoint 1: Scope & Data Architecture (Day 1)**

Never start a feature by prompting an AI to "build the UI." Begin at the data layer. Define

the exact PostgreSQL/Supabase schema, necessary indexing, relationship mapping, and backend routing. If the data architecture is flawed, the entire feature will eventually require a rewrite.

- **Checkpoint 2: Core Logic & Edge Cases (Day 2)**

Implement the backend controllers and frontend integration. Focus heavily on state management, asynchronous operations, and error handling. What happens if the API fails? What happens on a slow 3G connection? "Vibe coding" ignores errors; feature engineering anticipates them.

- **Checkpoint 3: Stress Testing & Security Validation (Day 3)**

Do not move to the next shiny feature. Spend this time trying to break what you just built. Check for IDOR vulnerabilities, unauthorized data access, and broken UI states. Validate that the feature performs flawlessly before committing to the main branch.

**4. Anti-Pattern & Prevention Matrix**

| Vibe Coding Anti-Pattern | Root Cause in AI Development                                                   | Engineering Protocol & Remediation                                                                                    |
|--------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| The 80% Graveyard        | Jumping to a new prompt before handling API error states and loading UIs.      | <b>Strict Scope Lock:</b><br>Complete frontend, backend, and error handling for one feature before ideating the next. |
| "Spaghetti" Architecture | Asking AI to build full-stack features in a single prompt without DB planning. | <b>Data-First Execution:</b><br>Always finalize schemas and API contracts before generating React/Next.js components. |
| Security Blindspots      | Assuming the AI included Auth/RLS checks in the rapid UI generation.           | <b>Day-3 QA Enforcement:</b><br>Mandate manual stress testing for IDOR and token validation on every new endpoint.    |